Concurrency and Distributed Systems

Week 4 – Part 1: Java Memory Model and Visibility

Dr. Mohamad Aoude

Faculty of Engineering

May 11, 2026

Part 1 Roadmap

Why This Part Matters

Core question

When one thread writes a value, **when** does another thread see it?

In sequential programming, this question barely appears. In concurrent programming, it is one of the central correctness questions.

- A program may look logically correct
- The variable may truly be updated
- Yet another thread may continue seeing an older value

Main message

Concurrency is not only about **multiple threads running**. It is also about **what each thread is allowed to observe**.

From Week 3 to Week 4

Week 3

We studied mechanisms that control **entry into critical sections**:

- `synchronized`
- `ReentrantLock`

Week 4

Now we study the memory side of concurrency:

- visibility
- ordering
- the Java Memory Model
- why stale reads happen
- why visibility is different from atomicity

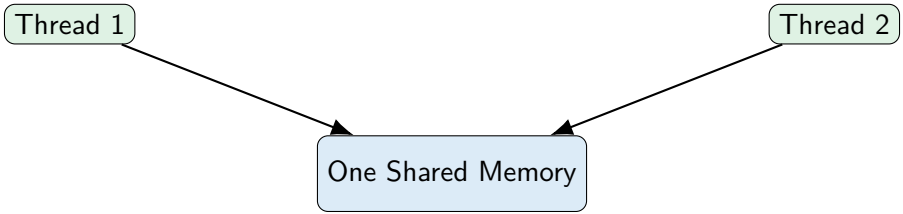
Learning Objectives

By the end of this lecture, students should be able to:

- explain why stale reads can happen
- describe, informally, the role of caches, registers, and reordering
- explain why Java needs a formal memory model
- distinguish between:
  - computation correctness in one thread
  - observation correctness across threads
- define **visibility** in practical terms
- identify an informal **data race**
- prepare for the use of `volatile` in the next lecture

The Naive Mental Model

Many students initially imagine shared memory like this:



Naive assumption

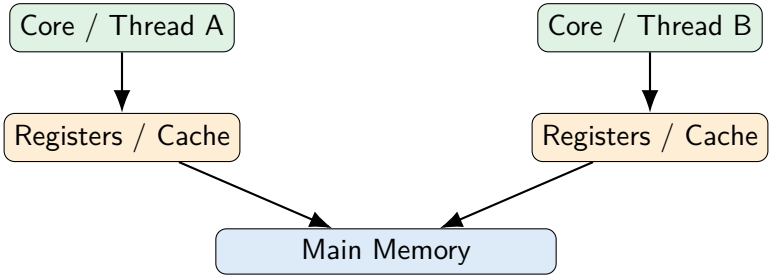
Every read goes directly to shared memory, and every write is instantly visible to everyone.

Problem

That is not how modern concurrent execution should be mentally modeled.

The Real System Is More Complicated

A more realistic picture includes multiple layers:



- values may be cached
- values may stay in registers
- reads may not observe the latest write immediately
- compiler and CPU may reorder operations

First Contradiction: The Stop-Flag Example

```
class StopDemo {
    static boolean running = true;

    public static void main(String[] args) throws
        Exception {
        Thread worker = new Thread(() -> {
            while (running) {
                // busy wait
            }
            System.out.println("Worker stopped");
        });

        worker.start();

        Thread.sleep(1000);
        running = false;
        System.out.println("Main changed running =
```

Question: Why Might This Fail?

At first glance, the code seems simple:

- main thread writes `running = false`
- worker thread repeatedly checks `running`
- therefore the worker should stop

But in concurrent execution

The worker may keep seeing an older value, such as `true`.

Possible reasons include:

- cached value reused
- value kept in register
- lack of synchronization semantics
- legal compiler/runtime optimizations

Important insight

Visibility: Informal Definition

Visibility

Visibility asks whether a write performed by one thread becomes observable by another thread.

Write correctness	Was the value updated at all?
Observation correctness	Did the other thread actually see that update?

Concurrency reality

A value can be correctly written, yet still not be promptly visible to another thread.

Engineering Intuition: Why Hardware Does This

Why are processors designed this way?

- accessing memory is expensive
- caches reduce latency
- registers reduce repeated loads
- CPUs reorder internally for performance
- compilers optimize aggressively when semantics allow it

Performance principle

Modern systems are optimized for throughput and efficiency, not for a naive line-by-line human mental model.

Key takeaway

Concurrency correctness requires **explicit memory semantics**. It cannot rely on intuition alone.

Single-Thread Semantics vs Multi-Thread Observation

Single-thread view

If a compiler reorders operations but preserves the meaning for one thread, the transformation may be allowed.

Multi-thread view

That same reordering may affect what another thread can observe, unless synchronization rules forbid it.

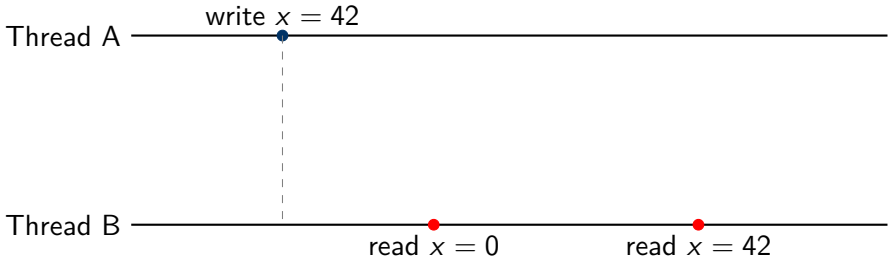
Important distinction

An optimization may be harmless for one thread but dangerous for shared-memory reasoning across threads.

This is why a formal memory model is necessary

Without it, developers would have no stable basis for reasoning about concurrent code.

A More Realistic Observation Timeline



What this means

Even after Thread A writes a new value, Thread B may not immediately observe that value in unsynchronized code.

Do not assume

`write now` does *not* imply `everyone sees it now`.

The Java Memory Model (JMM)

Definition

The Java Memory Model is the formal specification that defines how threads interact through memory in Java.

It provides rules about:

- what values reads are allowed to see
- when writes become visible
- which reorderings are legal
- how synchronization establishes guarantees

Why it matters

The JMM is the bridge between:

- Java source code
- compiler / JIT optimizations

Why Java Needs a Memory Model

Without a formal memory model:

- every processor could behave differently
- every JVM optimization could break assumptions differently
- concurrent code would become non-portable and non-reasonable

Practical answer

The JMM gives developers a contract:

If you use the right synchronization constructs, Java guarantees certain visibility and ordering properties.

Practical engineering viewpoint

The JMM does not promise "intuitive behavior." It promises **defined behavior**.

What This Lecture Covers vs Later Parts

Part 1	Why stale reads and visibility problems exist
Part 2	`volatile` and happens-before
Part 3	Visibility vs atomicity; broken DCL
Part 4	`AtomicInteger`, CAS, contention

Pedagogical logic

Before students can use `volatile`, they must first understand *why* unsynchronized code can fail.

Informal Data Race Idea

Informal definition

A data race occurs when:

- two or more threads access the same variable
- at least one access is a write
- there is no proper synchronization establishing safe ordering / visibility

Why data races matter

Once shared access is unsafely concurrent, behavior becomes much harder to reason about.

Important nuance

A data race is not just "two threads touching the same variable." It is unsafely touching it without the needed memory guarantees.

StopDemo as an Informal Data Race Example

```
1                    static boolean running = true;
```

Two threads access `running`:

- Thread A writes `running = false`
- Thread B repeatedly reads `running`

There is no:

- `volatile`
- monitor synchronization
- explicit lock-based communication

Conclusion

This is an unsafely shared variable communication pattern. It is exactly the kind of situation where stale reads may appear.

Why Adding `println` Can Change Behavior

A classic student surprise is this:

```
1                       while (running) {
2                               System.out.println("still running...");
3                       }
```

Now the program may appear to “work better.”

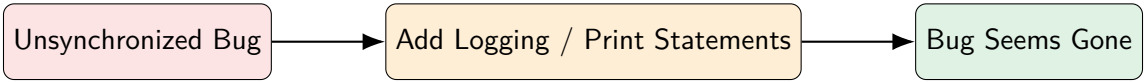
Why?

Because adding logging changes:

- timing
- scheduling
- optimization opportunities
- possibly synchronization side effects inside library code

Critical lesson

Debugging Trap: Observation Can Disturb the System



Engineering reality

Concurrency bugs are often:

- timing-sensitive
- schedule-dependent
- sensitive to instrumentation

Implication

Never conclude “it is fixed” just because it stops failing under observation.

Visibility Is Not the Same as Atomicity

This lecture is about **visibility**, but students must already see the boundary.

Visibility

Did one thread see another thread's update?

Atomicity

Did a multi-step operation behave as one indivisible action?

Why preview this now?

Students often confuse:

- "I can see the latest value"
- with
- "my update is thread-safe"

Part 2 and Part 3 will separate these carefully.

Three Core Questions of Shared Memory Concurrency

1. **Visibility:** Can one thread see what another thread wrote?
2. **Ordering:** Are operations observed in the required order?
3. **Atomicity:** Does a multi-step update behave as one unit?

Week 4 structure

- Part 1 emphasizes visibility intuition
- Part 2 introduces visibility + ordering with `volatile`
- Part 3 addresses atomicity and DCL
- Part 4 introduces atomics and CAS

A Practical Mental Model for Students

When reasoning about unsynchronized shared variables, do *not* assume:

- every read gets the newest global value
- every write becomes instantly visible to all threads
- source code order is always the observation order across threads

Instead, assume:

- threads may temporarily observe different views
- shared-memory communication requires proper semantics
- concurrency correctness must be deliberately engineered

Good engineering posture

In concurrent code, **assume nothing is safely communicated unless the model guarantees it**.

Mini Case Analysis

Case

Thread A computes:

$$done = true$$

Thread B spins:

```
while (!done) { }
```

Question: Why is this unsafe in plain shared-memory code?

Reasoning

Because the program gives no formal memory-visibility guarantee that the spinning thread must observe the update promptly.

This is the core of Part 1

The logic of the algorithm may be right, but the communication semantics are incomplete.

What Students Should Stop Saying After This Lecture

After Part 1, students should stop saying:

- "But I changed the variable, so the other thread must know"
- "It works on my machine, so the memory behavior is correct"
- "If the loop checks repeatedly, it will definitely see the new value"
- "Adding print statements fixed the concurrency bug"

Replace with

Without explicit memory semantics, another thread may legally observe stale state.

Concept Summary

Visibility	whether one thread observes another thread's write
Stale read	reading an older value after a newer one was written elsewhere
JMM	Java's formal rules for memory interaction across threads
Data race	unsafely concurrent access to shared data
Key risk	code may look right but still communicate incorrectly

Lab Preparation for Part 1

Lab 4.1 — Stale Read Demo

Students run the stop-flag example and observe its behavior under different modifications.

They should try:

- plain busy-wait loop
- adding `println`
- adding small work inside the loop
- repeating the run multiple times

Expected pedagogical outcome:

- behavior is unstable
- debugging changes timing
- the root issue is lack of proper visibility guarantees

Discussion Questions

1. Why is a shared variable not automatically a safe communication channel?
2. Why can two threads temporarily observe different values?
3. Why does adding output sometimes change concurrency behavior?
4. What problem does the Java Memory Model solve?
5. Why is stale visibility different from wrong arithmetic logic?

Takeaway

Final takeaway for Part 1

In concurrent programming, correctness depends not only on what was written, but also on what other threads are allowed to see.

Bridge to Part 2

Now that we understand *why* stale reads happen, the next question becomes:

How do we force visibility and ordering guarantees in Java?

That leads directly to:

`volatile` and happens-before

End of Part 1

Questions?